Java

Complete Learning Roadmap

Arvi's Personal Reference Guide • Beginner to Project-Ready

GitHub: aryansondharva/Java

Full Roadmap Overview

This is your complete Java journey. Don't skip phases — each builds on the previous one.

Phase	Topic Area	Topics Covered	Time
1	**Basics**	Setup, Hello World, Variables, Data Types, Operators, Input, Conditions, Loops, Arrays	2–3 weeks
2	**Methods & OOP**	Methods, Classes, Objects, Constructor, this, Inheritance, Polymorphism, Encapsulation, Abstraction	3–4 weeks
3	**Intermediate**	Strings, ArrayList, Collections, Exception Handling, File Handling	2–3 weeks
4	**Projects**	Calculator, Student System, Bank System, Mini Projects	Ongoing

Phase 1 — Basics

1. Setup

- Install JDK (Java Development Kit) — version 17 or 21 recommended
- Install VS Code + Extension Pack for Java
- Install Git from git-scm.com

★ **NOTE:** *JDK = compiler. Without it, Java won't run on your machine.*

2. First Program — Hello World

Every Java program has this exact structure:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

★ **NOTE:** *File name MUST match class name exactly. Hello.java → class Hello*

★ **NOTE:** *Java is case-sensitive. hello ≠ Hello ≠ HELLO*

★ **NOTE:** *main() is where your program starts every time*

- Run commands in terminal:
  - javac Hello.java ← compiles the file
  - java Hello ← runs the program

3. Variables & Data Types

Variables store data. Java needs you to declare the type before using it.

```
int age = 18;           // whole numbers
double price = 9.99;    // decimal numbers
char grade = 'A';       // single character
boolean isStudent = true;  // true or false
String name = "Arvi"; // text (capital S)
```

★ **NOTE:** *int, double, char, boolean — these are primitive types (lowercase)*

★ **NOTE:** *String is a class, not a primitive — always capital S*

★ **NOTE:** *Variable names: start with lowercase, no spaces (use camelCase)*

4. Operators

- Arithmetic: + - * / % (% = remainder/modulus)
- Comparison: == != > < >= <= (returns true/false)
- Logical: && (AND) || (OR) ! (NOT)
- Assignment: = += -= *= /=

★ **NOTE:** *== checks value equality, not identity (for objects use .equals())*

★ **NOTE:** *% is modulus — 10 % 3 = 1 (remainder after dividing)*

5. Input from User (Scanner)

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
int num = sc.nextInt();
String name = sc.nextLine();
```

★ **NOTE:** *Always import Scanner at the top of your file*

★ **NOTE:** *nextInt() for numbers, nextLine() for text, nextDouble() for decimals*

6. Conditions — if / else / switch

```
if (age >= 18) {
    System.out.println("Adult");
} else if (age >= 13) {
    System.out.println("Teen");
} else {
    System.out.println("Child");
}
```

★ **NOTE:** *switch is used when you have many fixed options (like a menu)*

7. Loops

- for loop — when you know how many times

```
for (int i = 0; i < 5; i++) { }
```

- while loop — when you don't know how many times

```
while (condition) { }
```

- do-while — runs at least once, then checks condition

```
do { } while (condition);
```

★ **NOTE:** *for loop: init ; condition ; update — all 3 parts*

★ **NOTE:** *break exits the loop. continue skips to next iteration*

8. Arrays

```
int[] marks = new int[5];          // empty array of 5
int[] marks = {90, 85, 78, 92, 88}; // with values
marks[0] = 95;   // access by index (starts at 0)
marks.length     // gives size of array
```

★ **NOTE:** *Array index starts from 0, not 1*

★ **NOTE:** *Array size is fixed — you can't add more elements later*

Phase 2 — Methods & OOP

9. Methods

```
public static int add(int a, int b) {
    return a + b;
}
```

- return type — what the method gives back (void = nothing)
- parameters — inputs to the method
- return — sends value back to caller

★ **NOTE:** *void means the method returns nothing*

★ **NOTE:** *Method name: starts with lowercase, camelCase*

10. Classes & Objects

```
class Car {
    String brand;
    int speed;
    void drive() { System.out.println("Driving"); }
}
Car myCar = new Car();  // creating object
myCar.brand = "Toyota";
```

★ **NOTE:** *Class = blueprint. Object = actual thing made from blueprint*

★ **NOTE:** *Use 'new' keyword to create an object*

11. Constructor

```
class Car {
    String brand;
    Car(String b) {   // constructor
        brand = b;
    }
}
Car c = new Car("Honda");
```

★ **NOTE:** *Constructor name = class name exactly*

★ **NOTE:** *Constructor runs automatically when object is created*

12. this Keyword

★ **NOTE:** *'this' refers to the current object*

```
Car(String brand) {
    this.brand = brand;  // this.brand = field, brand = parameter
}
```

13. Inheritance

```
class Animal {
    void eat() { System.out.println("Eating"); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}
```

★ **NOTE:** *extends = inheritance. Dog gets all Animal methods*

★ **NOTE:** *Java supports single inheritance only (one parent class)*

★ **NOTE:** *super keyword calls parent class methods/constructor*

14. Polymorphism

★ **NOTE:** *Same method, different behavior depending on object*

- Method Overloading — same name, different parameters (compile time)
- Method Overriding — child changes parent method (runtime)

```
// Overriding
@Override
void eat() { System.out.println("Dog eating"); }
```

15. Encapsulation

★ **NOTE:** *Hide data using private. Access using getters/setters*

```
private String name;
public String getName() { return name; }
public void setName(String n) { name = n; }
```

16. Abstraction & Interface

- Abstract class — partial implementation, use extends
- Interface — full abstraction, use implements

★ **NOTE:** *abstract method has no body — child class MUST implement it*

★ **NOTE:** *A class can implement multiple interfaces (unlike extends)*

⚙ Phase 3 — Intermediate

17. String Methods

```
String s = "Hello World";
s.length()          // 11
s.toUpperCase()     // HELLO WORLD
s.toLowerCase()     // hello world
s.charAt(0)         // H
s.contains("World") // true
s.substring(0, 5)   // Hello
s.replace("World", "Java") // Hello Java
s.split(" ")        // ["Hello", "World"]
```

★ **NOTE:** *Strings are immutable — methods return NEW strings, don't modify original*

★ **NOTE:** *Use .equals() to compare strings, not ==*

18. ArrayList & Collections

```
import java.util.ArrayList;
ArrayList<String> list = new ArrayList<>();
list.add("Arvi");
list.get(0);        // get by index
list.remove(0);     // remove by index
list.size();        // number of elements
```

★ **NOTE:** *ArrayList is dynamic — size grows automatically (unlike arrays)*

★ **NOTE:** *Use <Type> in angle brackets — this is called Generics*

19. Exception Handling

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("This always runs");
}
```

★ **NOTE:** *try — risky code. catch — handle error. finally — always runs*

★ **NOTE:** *Never leave catch block empty — always handle or print the error*

20. File Handling

```
import java.io.*;
FileWriter fw = new FileWriter("file.txt");
fw.write("Hello");
fw.close();  // Always close files
```

★ **NOTE:** *Always close files after use — use try-with-resources for safety*

🔧 Git Commands — Daily Use

Use these every day to save your progress to GitHub.

Command	What it does
`git init`	Start tracking a new project with Git
`git clone <url>`	Download a repo from GitHub to your computer
`git status`	See which files are changed / not saved yet

`git add .`	Stage ALL changed files (ready to save)
`git add filename.java`	Stage only ONE specific file
`git commit -m "message"`	Save staged files with a description
`git push`	Upload commits to GitHub
`git pull`	Download latest changes from GitHub
`git log --oneline`	See all your past commits (short form)
`git branch`	See all branches
`git checkout -b name`	Create and switch to a new branch

Daily Workflow (Do this every day!)

1. Write your Java code in VS Code
2. `Open terminal → type: git status`
3. `git add .`
4. `git commit -m "Day X: learned loops"`
5. `git push`

★ **NOTE:** *Commit message should describe WHAT you learned today*

★ **NOTE:** *Push every day — your GitHub becomes your progress diary*

⚡ Quick Reference Cheatsheet

Print Statements

```
System.out.println("text");  // prints + new line
System.out.print("text");    // prints, no new line
System.out.printf("%.2f", 3.14159); // formatted output
```

Data Type Sizes

- byte — 1 byte | short — 2 bytes | int — 4 bytes | long — 8 bytes
- float — 4 bytes | double — 8 bytes
- char — 2 bytes | boolean — 1 bit

★ **NOTE:** *Use int by default for whole numbers. Use double for decimals.*

★ **NOTE:** *Add L for long: 100L Add f for float: 3.14f*

Type Casting

```
int x = (int) 9.99;    // double to int → 9 (loses decimal)
double y = (double) 5 / 2; // → 2.5
```

★ **NOTE:** *Widening = automatic (int → double). Narrowing = manual cast needed.*

💡 Tips for Arvi

- Code every day — even 20 minutes is enough
- Push to GitHub daily — it shows your streak
- Don't memorize syntax — understand the logic
- When stuck, read the error message carefully
- Google is your friend — every developer uses it
- Build projects — they teach more than tutorials

★ **NOTE:** *The *** marks in your notes = important exam points*

★ **NOTE:** *Ask me ANYTHING — no question is too basic*

You got this, Arvi! ☕🚀